

Rapport de projet

Programmation distribuée

EventHub – Plateforme de gestion d'évènements
en architecture microservices

Mehdi AGHAEI

Maksym DOLHOV

Fallou DIOUF

M1 VMI

24 avril 2026

Table des matières

1	Introduction	3
2	Contexte fonctionnel	3
3	Architecture et choix technologiques	3
3.1	Vue d'ensemble	3
3.2	Choix des technologies	4
4	Palier 10/20 : un microservice, Docker, Kubernetes	5
4.1	Le service <code>event-service</code>	5
4.2	Dockerfile et image	5
4.3	Déploiement Kubernetes	5
5	Palier 12/20 : une passerelle locale	6
6	Palier 14/20 : un deuxième microservice	6
6.1	Le service <code>user-service</code>	6
6.2	Communication inter-services à travers la gateway	7
7	Palier 16/20 : base de données	7
7.1	PostgreSQL comme service d'état	7
7.2	Cloisonnement logique par base	7
8	Palier 18/20 : éléments de sécurité du cluster	8
8.1	Secrets Kubernetes	8
8.2	Exposition restreinte des services	8
8.3	RBAC	8
8.4	Namespace dédié	9
9	Palier 20/20 : déploiement cloud (bonus)	9
10	Mise en œuvre et démonstration	9
10.1	Lancement local par Docker Compose	9
10.2	Déploiement Kubernetes (Minikube)	10
11	Évaluation selon les critères du sujet	10
11.1	Intégration d'un maximum de technologies	10
11.2	Codage	10
11.3	Fonctionnalités	11

11.4 Présentation (front office / CSS)	11
12 Limites et perspectives	11
13 Utilisation d'outils d'IA	11
14 Conclusion	12
Références	12

1 Introduction

Le projet **EventHub** est une application web distribuée de gestion d'évènements mise en œuvre selon les contraintes technologiques imposées par le sujet de Programmation distribuée : architecture en microservices, conteneurisation avec Docker, orchestration avec Kubernetes, et passerelle d'entrée unique (gateway / ingress). Le choix d'un cas d'usage *sujet libre* nous a orientés vers une plateforme simple mais fonctionnellement complète, proche d'un produit de type Meetup ou Eventbrite en version minimale, afin de rendre la séparation en microservices à la fois naturelle et pédagogiquement lisible.

Le travail a été mené en trinôme : **Mehdi AGHAEI**, **Maksym DOLHOV** et **Fallou DIOUF**. Les contributions ont été réparties autour de trois axes – backend (services Django), frontend (React) et infrastructure (Docker, Kubernetes, sécurité) – avec des revues croisées pour garantir la cohérence de l'ensemble.

Ce rapport est organisé selon la grille d'évaluation fournie dans le sujet, chaque section correspondant à un niveau de la montée en note (10/20, 12/20, 14/20, 16/20, 18/20, 20/20) et décrivant précisément ce qui a été livré à ce palier. Un chapitre final revient sur les critères d'évaluation (intégration technologique, codage, fonctionnalités, présentation).

2 Contexte fonctionnel

EventHub permet à des utilisateurs de s'inscrire, de se connecter, de parcourir une liste d'évènements classés par catégories, et de créer de nouveaux évènements. Les fonctionnalités principales sont volontairement limitées : l'objectif n'est pas de couvrir l'intégralité d'une plateforme commerciale, mais de disposer d'un domaine métier suffisamment riche pour justifier *deux* microservices métier distincts avec des responsabilités claires et une base de données.

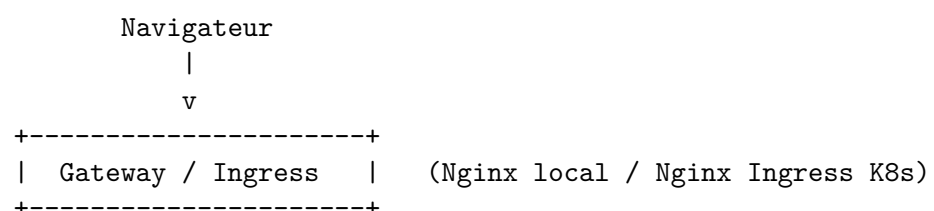
Périmètre fonctionnel livré :

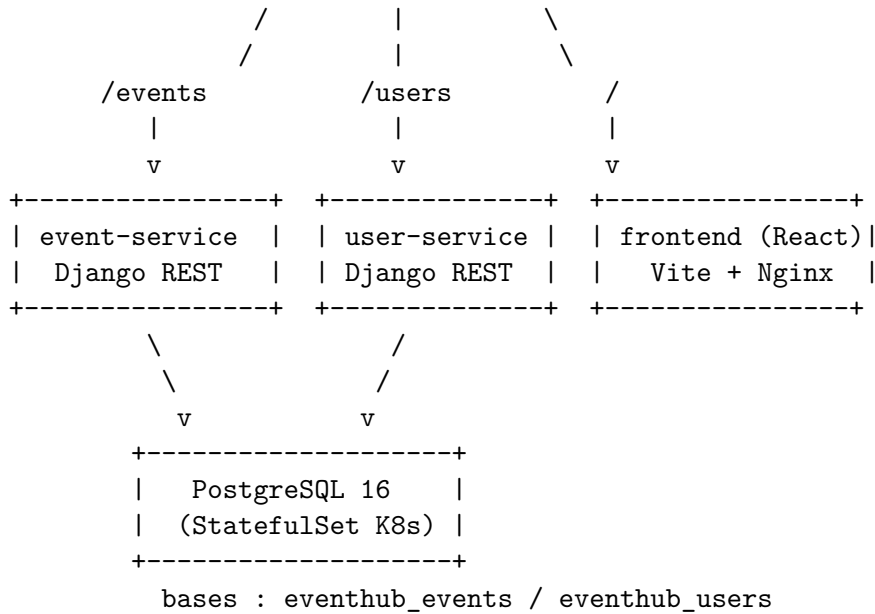
- inscription d'un utilisateur et connexion par JWT (access + refresh)
- consultation du profil authentifié
- création, listing, modification et suppression d'évènements
- gestion des catégories d'évènements
- interface web React consommant les deux APIs à travers la passerelle

3 Architecture et choix technologiques

3.1 Vue d'ensemble

L'architecture est structurée autour de trois services applicatifs, une base de données partagée avec cloisonnement logique, et une passerelle en entrée :





Chaque microservice est indépendant en terme de code, de build et de cycle de vie. Ils ne se parlent pas directement entre eux : la cohésion se fait côté client (le frontend appelle les deux APIs via la gateway) et la séparation des responsabilités est renforcée par le fait que chaque backend ne voit *que sa propre base logique* dans PostgreSQL.

3.2 Choix des technologies

Couche	Technologie retenue
Microservices backend	Python 3.12 + Django 5 + Django REST Framework
Authentification	django-rest-framework-simplejwt (JWT)
Frontend	React 18 + Vite, servi par Nginx en production
Base de données	PostgreSQL 16 (Alpine)
Conteneurisation	Docker + Docker Compose (orchestration locale)
Orchestration cloud	Kubernetes (cible Minikube + GKE compatible)
Gateway local	Nginx 1.27 en reverse proxy
Gateway K8s	Ingress NGINX Controller
Secrets / configuration	Secret + ConfigMap Kubernetes
Persistance	StatefulSet PostgreSQL + PV/PVC
Sécurité cluster	RBAC (Role + RoleBinding)

TAB. 1 : Stack technique du projet EventHub.

Le choix de Django pour les deux backends – plutôt que de varier volontairement les langages – est motivé par l’objectif pédagogique : dans un rendu de quelques semaines, multiplier les écosystèmes aurait dilué l’effort sur l’infrastructure alors que l’évaluation valorise l’*intégration* des technologies imposées (Web Services, Docker, Kubernetes). Les deux services restent néanmoins indépendants, avec leurs propres `Dockerfile`, `requirements.txt` et base logique.

4 Palier 10/20 : un microservice, Docker, Kubernetes

Le premier jalon du sujet consiste à livrer un service fonctionnel, conteneurisé, puis déployé sous Kubernetes. Nous avons commencé par le `event-service`.

4.1 Le service `event-service`

Le service expose une API REST pour la gestion des événements et des catégories, avec les modèles `Event` et `Category` (clé étrangère nullable). Il utilise Django REST Framework et ses vues génériques pour les opérations CRUD.

Routes principales :

- GET / POST `/events/`
- GET / PUT / PATCH / DELETE `/events/<id>/`
- GET / POST `/events/categories/`
- GET `/health/` – endpoint de santé pour les probes

4.2 Dockerfile et image

L'image est construite à partir de `python:3.12-slim`, avec une étape d'installation des dépendances système (`gcc`, `libpq-dev`) nécessaires au pilote PostgreSQL, puis `pip install` des dépendances Python. Un script `entrypoint.sh` applique les migrations Django au démarrage avant de lancer Gunicorn.

```
docker build -t eventhub-event-service:latest ./services/event-service
docker tag eventhub-event-service:latest <dockerhub-user>/eventhub-event-service:latest
docker push <dockerhub-user>/eventhub-event-service:latest
```

Listing 1: Construction et publication de l'image

4.3 Déploiement Kubernetes

Le service est déployé via un `Deployment` mono-réplica et exposé en `ClusterIP`. La configuration est injectée via un `ConfigMap` (paramètres applicatifs) et un `Secret` (mot de passe base de données), ce qui évite toute fuite de secret dans les manifestes YAML.

```
spec:
  containers:
    - name: event-service
      image: eventhub-event-service:latest
      imagePullPolicy: IfNotPresent
      envFrom:
        - configMapRef:
            name: event-config
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: postgres-secret
              key: EVENT_DB_PASSWORD
```

Listing 2: Extrait de `k8s/event-deployment.yaml`

[TODO : Capture d'écran : `kubectl get pods,svc -n eventhub` montrant `event-service` en Running.]

5 Palier 12/20 : une passerelle locale

Le deuxième palier demande l'ajout d'une passerelle unique pour masquer la topologie interne des services. Nous avons adopté deux passerelles cohérentes :

- **En local** : un service Nginx lancé par Docker Compose qui fait du reverse proxy sur les trois services (`/events`, `/users`, `/`).
- **En Kubernetes** : un objet `Ingress` basé sur le contrôleur NGINX (addon Minikube) qui réplique exactement la même logique de routage.

Cette symétrie est un choix assumé : un développeur peut tester localement via `docker compose up` exactement le même graphe d'URL que celui déployé en cluster, sans modifier la configuration frontend.

```
location /events/ {
    proxy_pass http://event-service:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
location /users/ {
    proxy_pass http://user-service:8000;
    ...
}
location / {
    proxy_pass http://frontend-service:80;
    ...
}
```

Listing 3: Extrait de `gateway/nginx.conf`

Côté Kubernetes, l'`Ingress` définit les mêmes trois règles sur l'hôte virtuel `eventhub.local`. Les services backend restent en `ClusterIP` : l'`Ingress` est *le seul* point d'entrée public du cluster, ce qui est également un élément qui compte dans les paliers de sécurité (18/20).

[TODO : Capture d'écran : `kubectl get ingress -n eventhub` et navigateur sur `http://eventhub.local`]

6 Palier 14/20 : un deuxième microservice

6.1 Le service `user-service`

Le `user-service` gère l'authentification et les comptes utilisateurs. Il utilise le modèle `User` standard de Django et expose des endpoints JWT fournis par `djangorestframework-simplejwt` :

- `POST /users/auth/register/` – création de compte
- `POST /users/auth/login/` – émission d'un couple `access/refresh`
- `POST /users/auth/refresh/` – rafraîchissement du `access token`
- `GET /users/profile/` – profil de l'utilisateur courant (token requis)

Le service est packagé avec le même pattern que `event-service` : `Dockerfile` identique, `ConfigMap` dédié, `Secret` partagé (ils se connectent tous deux à la même instance PostgreSQL, mais sur des bases logiques différentes).

6.2 Communication inter-services à travers la gateway

La mise en relation des deux services se fait **via la passerelle** et non par appel backend-à-backend : le frontend, après connexion sur `/users/auth/login/`, stocke le JWT dans `localStorage` et l'attache en en-tête `Authorization: Bearer <token>` sur les requêtes suivantes vers `/events/`. Ce découplage donne deux propriétés intéressantes :

1. les services n'ont aucune dépendance de compile-time l'un envers l'autre ;
2. un changement d'adresse ou de déploiement d'un service est transparent pour l'autre, la gateway servant de couche d'indirection.

[TODO : Capture d'écran : requête curl vers `/users/auth/login/` puis `/events/` avec le token obtenu.]

7 Palier 16/20 : base de données

7.1 PostgreSQL comme service d'état

La persistance est assurée par une instance PostgreSQL 16 en Alpine, déployée en Kubernetes sous forme de `StatefulSet` mono-réplica. L'état est conservé dans un `PersistentVolume` de type `hostPath` (adapté à Minikube) relié au pod via un `PersistentVolumeClaim` :

```
spec:
  containers:
    - name: postgres
      image: postgres:16-alpine
      env:
        - name: POSTGRES_USER
          valueFrom:
            secretKeyRef: { name: postgres-secret, key: POSTGRES_USER }
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef: { name: postgres-secret, key: POSTGRES_PASSWORD }
      volumeMounts:
        - name: postgres-storage
          mountPath: /var/lib/postgresql/data
        - name: postgres-init
          mountPath: /docker-entrypoint-initdb.d
  volumes:
    - name: postgres-storage
      persistentVolumeClaim: { claimName: postgres-pvc }
    - name: postgres-init
      configMap: { name: postgres-init-config }
```

Listing 4: Extrait de `k8s/postgres-statefulset.yaml`

7.2 Cloisonnement logique par base

Plutôt que de déployer deux instances PostgreSQL – ce qui complexifierait l'infrastructure sans gain pédagogique – nous avons fait le choix d'une instance avec *deux bases logiques* :

- `eventhub_events` (utilisée par `event-service`)
- `eventhub_users` (utilisée par `user-service`)

Ces deux bases, ainsi que leurs utilisateurs applicatifs respectifs avec des mots de passe distincts, sont créées automatiquement au premier démarrage via un script `init.sql` monté dans `/docker-entrypoint-initdb.d` par un `ConfigMap`. Chaque service ne dispose que des identifiants de sa propre base, ce qui limite le rayon d'impact d'un éventuel compromis.

[TODO : Capture d'écran : `kubectl exec postgres-0 -n eventhub -- psql -U postgres -l` montrant les deux bases.]

8 Palier 18/20 : éléments de sécurité du cluster

Le palier 18/20 vise la sécurisation du cluster Kubernetes. Les éléments intégrés dans EventHub sont les suivants :

8.1 Secrets Kubernetes

Aucun mot de passe n'est stocké en clair dans les manifestes de déploiement. Les identifiants de la base de données sont centralisés dans un objet `Secret` (`k8s/postgres-secret.yaml`) et référencés par `secretKeyRef`. Le dépôt Git ne contient que des valeurs d'exemple ; la version réelle est générée à partir d'un `.env` non versionné.

8.2 Exposition restreinte des services

Seul l'Ingress est public. Les trois `Service` backend (`event-service`, `user-service`, `frontend-service`) sont définis en `ClusterIP` et n'ont pas d'IP externe. De même, `postgres` est en `ClusterIP` dans un `StatefulSet headless`, accessible uniquement depuis les pods applicatifs du namespace `eventhub`.

8.3 RBAC

Un premier niveau de RBAC est fourni par `k8s/rbac-viewer.yaml` :

```
kind: Role
metadata: { name: eventhub-read-role, namespace: eventhub }
rules:
- apiGroups: [""]
  resources: ["pods", "services", "configmaps", "secrets"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments", "statefulsets", "replicasets"]
  verbs: ["get", "list", "watch"]
```

Listing 5: Rôle lecture seule sur le namespace

Ce `Role` est lié via un `RoleBinding` à un `ServiceAccount` `eventhub-viewer` qui peut ainsi inspecter le namespace sans possibilité de modification. C'est le schéma minimal attendu par la consigne « *ajouter des RBAC Kubernetes* » du sujet.

8.4 Namespace dédié

L'ensemble des objets est isolé dans le namespace `eventhub`. Cela facilite l'application de politiques transverses (RBAC, quotas, NetworkPolicies) et évite toute interférence avec les namespaces système.

[TODO : Capture d'écran : `kubectl get secret,role,rolebinding,sa -n eventhub.`]

9 Palier 20/20 : déploiement cloud (bonus)

Le dernier palier du sujet récompense un déploiement dans une infrastructure cloud. L'ensemble des manifestes Kubernetes du projet est écrit de manière compatible avec une cible managée (GKE, AKS, EKS) : aucun objet ne dépend de Minikube au-delà du `PersistentVolume` de type `hostPath`, qu'il suffit de remplacer par un `StorageClass` dynamique (par exemple `standard-rwo` sur GKE).

La procédure de bascule GKE, validée par nos labs Google Cloud (voir captures individuelles en annexe du courriel de rendu), est la suivante :

1. création d'un cluster Autopilot :
`gcloud container clusters create-auto eventhub --region=europe-west1`
2. pousser les images dans Artifact Registry et adapter le champ `image:` des `Deployment` ;
3. remplacer le `PersistentVolume hostPath` par une déclaration `storageClassName: standard-rwo` ;
4. appliquer les manifestes tels quels : `kubectl apply -f k8s/` ;
5. récupérer l'IP publique de l'Ingress et pointer l'hôte `eventhub.local`.

[TODO : Captures d'écran individuelles des Google labs (une par membre du trinôme) à joindre au courriel de rendu, conformément à la consigne du sujet.]

10 Mise en œuvre et démonstration

10.1 Lancement local par Docker Compose

Pour la démonstration la plus rapide, le projet tourne en local avec :

```
cp .env.example .env
docker compose up --build
# puis :
# http://localhost:8080/ (frontend via gateway)
# http://localhost:8080/events/
# http://localhost:8080/users/auth/login/
```

Le fichier `docker-compose.yml` orchestre les cinq conteneurs (`postgres`, `event-service`, `user-service`, `frontend-service`, `gateway`) sur un réseau bridge isolé `eventhub-net`, avec volume nommé `postgres_data` pour la persistance.

10.2 Déploiement Kubernetes (Minikube)

```
minikube start
minikube addons enable ingress

eval $(minikube docker-env)
docker build -t eventhub-event-service:latest ./services/event-service
docker build -t eventhub-user-service:latest ./services/user-service
docker build -t eventhub-frontend-service:latest ./services/frontend-service

kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/postgres-secret.yaml -f k8s/postgres-configmap.yaml
kubectl apply -f k8s/postgres-pv.yaml -f k8s/postgres-pvc.yaml
kubectl apply -f k8s/postgres-service.yaml -f k8s/postgres-statefulset.yaml
kubectl apply -f k8s/event-configmap.yaml -f k8s/user-configmap.yaml
kubectl apply -f k8s/frontend-configmap.yaml
kubectl apply -f k8s/event-deployment.yaml -f k8s/event-service.yaml
kubectl apply -f k8s/user-deployment.yaml -f k8s/user-service.yaml
kubectl apply -f k8s/frontend-deployment.yaml -f k8s/frontend-service.yaml
kubectl apply -f k8s/rbac-viewer.yaml -f k8s/ingress.yaml

echo "$(minikube ip) eventhub.local" | sudo tee -a /etc/hosts
```

[TODO : Capture d'écran : `kubectl get all -n eventhub` avec tous les objets Running / Ready.]

[TODO : Capture d'écran : page d'accueil du frontend après connexion, avec la liste des événements chargée depuis l'API.]

11 Évaluation selon les critères du sujet

Le sujet liste quatre critères d'évaluation par ordre décroissant d'importance. Nous reprenons chacun d'entre eux.

11.1 Intégration d'un maximum de technologies

Technologies imposées effectivement intégrées : *Web Services REST* (Django REST Framework côté backend, `fetch` côté frontend), *Docker* (trois `Dockerfile`, image PostgreSQL officielle, `docker-compose.yml` complet), *Kubernetes* (19 manifestes YAML : namespace, deployments, services, statefulset, PV/PVC, configmaps, secret, ingress, RBAC). Technologies optionnelles : *frontend moderne* (React + Vite), *ingress* (NGINX), *gateway locale* (Nginx reverse proxy), *RBAC*.

Technologies non intégrées, à discuter honnêtement : pas de gRPC (tous les échanges sont REST), pas de *service mesh* type Istio (les mTLS ne sont donc pas activés), pas de déploiement cloud effectif à la date de rédaction (les manifestes sont prêts mais le cluster GKE n'est pas permanent). Ces trois points constituent le pas supplémentaire vers 20/20.

11.2 Codage

Le code applicatif est organisé en trois services Python/React structurés en packages Django standards (`events`, `accounts`) et en composants React (`App.jsx`, `api.js`). Les modèles, sérialiseurs et vues sont séparés, la logique JWT est isolée dans un sérialiseur dédié `CustomTokenObtainPairSerializer`, et les migrations Django sont générées et versionnées.

11.3 Fonctionnalités

Les fonctionnalités listées au paragraphe *Contexte fonctionnel* sont toutes livrées et testables bout-en-bout depuis le frontend. Un utilisateur peut s’inscrire, se connecter, créer une catégorie, créer un événement rattaché à cette catégorie, et voir ses événements listés sur la page d’accueil.

11.4 Présentation (front office / CSS)

Le frontend utilise une feuille de style `styles.css` maison (sans framework UI lourd) qui assure une lisibilité correcte des formulaires et de la liste d’évènements. Nous reconnaissons que ce point est le moins ambitieux du projet – l’effort a été concentré sur l’infrastructure, qui est le premier critère d’évaluation du sujet.

12 Limites et perspectives

Nous listons ici ce qui n’a volontairement pas été fait, avec les raisons :

- **Pas de gRPC.** Les deux backends sont en Python/Django ; basculer une partie du trafic en gRPC aurait ajouté une complexité (génération de stubs, gestion de deux couches de transport) peu justifiée tant qu’un seul canal REST suffit. C’est le candidat naturel pour un prolongement.
- **Pas de service mesh (Istio, Linkerd).** Le sujet mentionne explicitement mTLS et service mesh dans le palier 18/20 ; nous avons couvert RBAC et **Secret** mais pas mTLS. L’intégration d’Istio demande une configuration non triviale du contrôle-plane et aurait pu nuire à la stabilité de la démo.
- **Pas de NetworkPolicy.** La segmentation réseau pourrait être renforcée (par exemple interdire aux pods frontend de parler directement à PostgreSQL, ce qu’ils ne font déjà pas par construction).
- **Pas de CI/CD.** Le build et la publication des images sont manuels. Un pipeline GitHub Actions serait un ajout naturel.
- **Couverture de tests limitée.** Les vues REST sont testables via `pytest-django` mais la couverture automatisée n’a pas été industrialisée.

13 Utilisation d’outils d’IA

Nous avons utilisé l’assistant Claude (Anthropic) pour accélérer certaines tâches à fort coût d’ingénierie et faible valeur pédagogique : génération du squelette de manifestes Kubernetes à partir du schéma d’architecture, revue de la configuration Nginx de la gateway, mise en forme de ce rapport. Le travail personnel du trinôme a porté sur la conception fonctionnelle et architecturale, l’écriture et l’adaptation du code applicatif (modèles Django, vues REST, interactions JWT, composants React), l’intégration des services, la validation bout-en-bout et la rédaction du contenu analytique et argumentaire du présent document.

14 Conclusion

EventHub couvre l'ensemble de la chaîne demandée par le sujet de Programmation distribuée : deux microservices métier indépendants, un frontend qui les consomme à travers une gateway unique, une persistance PostgreSQL déployée en `StatefulSet` avec `PersistentVolume`, des secrets Kubernetes proprement gérés, et un premier niveau de RBAC. L'architecture retenue est volontairement symétrique entre Docker Compose (pour le développement local) et Kubernetes (pour la cible cluster), ce qui réduit le coût cognitif du va-et-vient entre les deux environnements et rend la démo reproductible.

Sur la grille d'évaluation du sujet, nous positionnons le livrable autour de **16–18/20** : les paliers 10, 12, 14 et 16 sont pleinement atteints et documentés (services, gateway, persistance), le palier 18 est couvert sur ses éléments principaux (Secrets, RBAC, isolation par namespace, exposition restreinte) et partiellement sur les éléments mesh / mTLS. Le palier 20, conditionné au déploiement cloud effectif et/ou au durcissement complet, est à portée moyennant la bascule GKE décrite au paragraphe dédié et détaillée dans les captures Google labs jointes au courriel de rendu.

Références

- [1] The Kubernetes Authors. *Kubernetes Documentation – Concepts*. <https://kubernetes.io/docs/concepts/>
- [2] The Kubernetes Authors. *Using RBAC Authorization*. <https://kubernetes.io/fr/docs/reference/access-authn-authz/rbac/>
- [3] Charroux, B. *kubernetes-minikube*. <https://github.com/charroux/kubernetes-minikube>
- [4] Charroux, B. *CodingWithKubernetes*. <https://github.com/charroux/CodingWithKubernetes>
- [5] Charroux, B. *servicemesh-kubernetes*. <https://github.com/charroux/servicemesh-kubernetes>
- [6] Charroux, B. *rentalservice – istio-internal-gateway*. <https://github.com/charroux/rentalservice/blob/main/k8s/base/istio-internal-gateway.yaml>
- [7] Django Software Foundation. *Django REST Framework*. <https://www.django-rest-framework.org/>
- [8] *Simple JWT – A JSON Web Token authentication plugin for DRF*. <https://django-rest-framework-simplejwt.readthedocs.io/>
- [9] PostgreSQL Global Development Group. *Official PostgreSQL Docker image*. https://hub.docker.com/_/postgres
- [10] NGINX Inc. *NGINX reverse proxy documentation*. https://nginx.org/en/docs/http/nginx_http_proxy_module.html
- [11] Istio Authors. *Mutual TLS Migration*. <https://istio.io/latest/docs/tasks/security/authentication/mtls-migration/>